

Building Framework-agnostic Web Components in Angular

In a tech environment filled with polyglots who're comfortable with various JavaScript frameworks (Angular, React, Vue, Ember, etc), let us understand how to develop reusable components and elements in Angular, which can be reused seamlessly across JavaScript frameworks.



As the old saying goes, the only constant in life is change. There are many JavaScript frameworks available today. These include Angular, React, Vue.js, Ember, and many more. Initially, we choose a single framework and build the whole application in a sort of monolithic way. However, the reality is that, over time, we must embrace the pluralism in frameworks. The product we are working on becomes, quite obviously, a polyglot (conversant in many languages). The impending challenge is to build a component (element) in one framework and seamlessly reuse it in others. A Web component is the immediate solution. It is novel, easy-to-implement, easy-to-maintain and flexible enough to be used across JavaScript ES2015 compatible frameworks.

In this article, we are going to use Angular as the base framework. We will look at how to implement a reusable, framework-agnostic Web component from scratch. After bundling it as a JavaScript file, we will deploy and use it as well.

Why Web components

Web components can be built once in your favourite JavaScript framework, yet be used in every other framework,

without any changes. Frameworks are not eternal; developers start with one and slowly move on to others. 'Framework polyglots' is the current reality. This is even more so with the ubiquitous presence of FOSS around us. The typical shelf life of a front-end framework in a product is around five years or so. After which another rich framework shows up, and the developers and community hop onto it merrily. There's this eternal quest to move to something shinier, newer and better. There are, of course, many factors that influence such a move. Some are organic, whereas some are purely inorganic.

Organic factors: The product you're working on merges or integrates with another product. The need to be a polyglot is organic in this case.

Inorganic: A company acquires another, or gets acquired, paving the way for inorganic polyglotism.

The mantra for staying afloat in such a multi-platform world is to build the elements/components once and let them be reused across similar frameworks seamlessly throughout. This is the reason for the advent and proliferation of Web components in today's dynamic front-end Web frameworks.

Let us now dive into some technicalities.

Installing Angular

Angular is one of the most popular JavaScript front-end frameworks. In this section, let us look at how to install the latest version of Angular (Angular 9 was released in November 2019).

To begin, give the following command:

```
npm install -g @angular/cli
```

This installs Angular CLI's latest version. Now type the following command:

```
ng new angular-web-component
```

This creates a new project under the \$pwd/angular-web-component directory. Now give the following command:

```
cd angular-web-component
ng add @angular/elements
```